

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: ITA 06

COURSE NAME: MACHINE LEARNING

COURSE OUTCOME COVERED

CO4: K- Nearest Neighbour Learning – Locally weighted Regression – Radial Bases Functions – Case Based Learning **(BL 5)**

Assessment Tool Weightage: 40%

QUESTIONS: 15

Total Marks:25

Annexure A

Coding Challenge

Code of Conduct:

I D. Sindhu (192424197) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.



Signature of the student:

Annexure A

Short Answer Test

- 1, Write a Python program to implement the K-Nearest Neighbour (KNN) algorithm for a small dataset. Use Euclidean distance to find nearest neighbors and classify a test instance. Allow the user to input the value of K and display predicted class labels with accuracy.
2. Develop a Python program to implement Locally Weighted Regression. Given a dataset, compute weights based on distance and predict output for a query point. Use a simple kernel function and compare predictions with standard linear regression for better understanding.
3. Write a Python program to implement a simple Radial Basis Function (RBF) network for regression or classification. Use Gaussian functions as activation units and train the model on a small dataset. Display predicted outputs and analyze how centers and widths affect performance.
4. Create a Python program that implements a basic Case-Based Learning system. Store past cases in a dataset and retrieve the most similar case for a new query using distance measures. Output the solution based on the closest match and explain similarity calculation.

5. Write a Python program to implement both KNN and RBF models on the same dataset. Compare their predictions for given test inputs. Analyze differences in computation, accuracy, and output behavior, and display results clearly for better comparison.
6. Write a Python program to implement the **Decision Tree algorithm** for classification. Use a small dataset and construct the tree using information gain or Gini index. Allow prediction for test instances and display accuracy. Visualize the tree structure and analyze how feature selection impacts classification performance.
7. Develop a Python program to implement a **Naïve Bayes classifier** for a given dataset. Compute prior and conditional probabilities, and classify a test instance. Display predicted class labels and evaluate performance using accuracy. Compare results with and without Laplace smoothing and analyze its impact on classification.
8. Write a Python program to implement **Logistic Regression** for binary classification. Train the model using gradient descent and predict outputs for test data. Display the sigmoid function output and classification results. Evaluate the model using accuracy and discuss how learning rate affects convergence and performance.
9. Create a Python program to implement the **K-Means clustering algorithm**. Initialize cluster centers, assign points to clusters, and update centroids iteratively. Display final clusters and visualize them using plots. Analyze how the number of clusters and initialization methods affect clustering results.
10. Write a Python program to implement a simple **Genetic Algorithm** to optimize a mathematical function. Initialize a population, perform selection, crossover, and mutation operations, and evolve solutions over generations. Display fitness values and best solution, and analyze how parameters influence convergence speed and solution quality.
11. Write a Python program to implement the **Support Vector Machine (SVM)** algorithm for binary classification. Use a simple dataset and implement a linear kernel. Train the model, classify test instances, and display accuracy. Analyze how the margin and support vectors influence classification performance.
12. Develop a Python program to implement **Principal Component Analysis (PCA)** for dimensionality reduction. Apply PCA on a dataset, reduce it to two principal components, and visualize the transformed data. Compare model performance before and after dimensionality reduction and analyze variance retention.
13. Write a Python program to implement the **Hierarchical Clustering algorithm**. Use a small dataset and apply different linkage methods such as single, complete, and average linkage. Generate a dendrogram and analyze how cluster formation changes with different linkage criteria.
14. Create a Python program to implement a simple **Artificial Neural Network (ANN)** with one hidden layer. Train the network using forward propagation and backpropagation. Test the model on sample inputs and display predicted outputs. Analyze how the number of neurons and learning rate affect performance.
15. Write a Python program to implement the **Apriori algorithm** for association rule mining. Use a transaction dataset to generate frequent itemsets and association rules. Display support, confidence, and lift values, and analyze how minimum support and confidence thresholds affect the results.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	30	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	20	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	15	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	20	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	15	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand

1. Answer

```

import math
from collections import Counter
X = [[1, 2], [2, 3], [3, 1], [6, 5], [7, 7], [8, 6]]
y = [0, 0, 0, 1, 1, 1]
test = [5, 5]
k = int(input("Enter K: "))
distances = []
for i in range(len(X)):
    d = math.sqrt((X[i][0] - test[0])**2 +
                  (X[i][1] - test[1])**2)
    distances.append((d, y[i]))
distances.sort()
nearest = distances[:k]
labels = [label for d, label in nearest]
prediction = Counter(labels).most_common(1)[0][0]
correct = 0
for i in range(len(X)):
    d = math.sqrt((X[i][0] - test[0])**2 +
                  (X[i][1] - test[1])**2)
    if y[i] == prediction:
        correct += 1

```

```

accuracy = correct / len(X) * 100
print("Predicted Class:", prediction)
print("Accuracy:", accuracy, "%")

```

2. Answer

```

import numpy as np
from sklearn.linear_model import LinearRegression
X = np.array([1, 2, 3, 4, 5])
y = np.array([2, 4, 5, 8, 10])
query = 3.5
tau = 1.0
weights = np.exp(-(X - query) ** 2) / (2 * tau ** 2)
model = LinearRegression()
model.fit(X.reshape(-1, 1), y, sample_weight=weights)
prediction = model.predict([[query]])
print("LWR Prediction:", prediction[0])
model2 = LinearRegression()
model2.fit(X.reshape(-1, 1), y)
print("Linear Regression:", model2.predict([[query]])[0])

```

3. Answer

```

import numpy as np
X = np.array([1, 2, 3, 4, 5])
y = np.array([0, 0, 1, 1, 1])
centers = np.array([2, 4])
sigma = 1.0
def rbf(x, center):
    return np.exp(-(x - center) ** 2) / (2 * sigma ** 2)
A = []
for x in X:
    A.append([rbf(x, c) for c in centers])
A = np.array(A)
weights = np.linalg.lstsq(A, y, rcond=None)[0]
test = 3
features = np.array([rbf(test, c) for c in centers])
prediction = np.dot(features, weights)
print("Prediction:", prediction)
if prediction >= 0.5:
    print("Class: 1")
else:
    print("Class: 0")

```

4. Answer

```

import math
cases = [
    ([20, 80], "Approved"),
    ([25, 85], "Approved"),
    ([10, 50], "Rejected"),
    ([12, 55], "Rejected")
]

```

```

query = [22, 82]
best_distance = float("inf")
best_solution = None
for features, solution in cases:
    distance = math.sqrt(
        (features[0] - query[0])**2 +
        (features[1] - query[1])**2
    )
    if distance < best_distance:
        best_distance = distance
        best_solution = solution
print("Closest Case Solution:", best_solution)
print("Distance:", best_distance)

```

5. Answer

```

from sklearn.neighbors import KNeighborsClassifier
from sklearn.neural_network import MLPClassifier
X = [[1], [2], [3], [6], [7], [8]]
y = [0, 0, 0, 1, 1, 1]
test = [[5]]
# KNN
knn = KNeighborsClassifier(n_neighbors=3)
knn.fit(X, y)
knn_prediction = knn.predict(test)
# RBF-like model using hidden layer
rbf = MLPClassifier(hidden_layer_sizes=(5,), max_iter=2000,
                    random_state=1)
rbf.fit(X, y)
rbf_prediction = rbf.predict(test)
print("KNN Prediction:", knn_prediction)
print("RBF Prediction:", rbf_prediction)

```

6. Answer

```

from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score
from sklearn.tree import plot_tree
import matplotlib.pyplot as plt
X = [
    [90, 80],
    [85, 75],
    [70, 60],
    [60, 55],
    [95, 90],
    [65, 50]
]
y = [1, 1, 0, 0, 1, 0]
model = DecisionTreeClassifier(criterion="gini", random_state=1)
model.fit(X, y)
test = [[80, 70]]
print("Prediction:", model.predict(test))

```

```

predictions = model.predict(X)
print("Accuracy:", accuracy_score(y, predictions))
plot_tree(model, filled=True)
plt.show()

```

7. Answer

```

from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score
X = [
    [1, 20],
    [2, 25],
    [3, 30],
    [8, 70],
    [9, 80],
    [10, 90]
]
y = [0, 0, 0, 1, 1, 1]
model = GaussianNB()
model.fit(X, y)
test = [[5, 50]]
print("Prediction:", model.predict(test))
predictions = model.predict(X)
print("Accuracy:", accuracy_score(y, predictions))
print("Probability:", model.predict_proba(test))

```

8. Answer

```

import numpy as np
X = np.array([1, 2, 3, 6, 7, 8], dtype=float)
y = np.array([0, 0, 0, 1, 1, 1], dtype=float)
w = 0
b = 0
learning_rate = 0.1
def sigmoid(z):
    return 1 / (1 + np.exp(-z))
for i in range(1000):
    z = w * X + b
    p = sigmoid(z)
    dw = np.mean((p - y) * X)
    db = np.mean(p - y)
    w = w - learning_rate * dw
    b = b - learning_rate * db
test = 5
probability = sigmoid(w * test + b)
if probability >= 0.5:
    prediction = 1
else:
    prediction = 0
print("Sigmoid Output:", probability)
print("Prediction:", prediction)

```

9. Answer

```

from sklearn.cluster import KMeans
import matplotlib.pyplot as plt
X = [
    [1, 2],
    [2, 3],
    [3, 2],
    [8, 8],
    [9, 9],
    [8, 9]
]
model = KMeans(n_clusters=2, random_state=1, n_init=10)
model.fit(X)
print("Cluster Labels:", model.labels_)
print("Centroids:", model.cluster_centers_)
plt.scatter(
    [x[0] for x in X],
    [x[1] for x in X],
    c=model.labels_
)
plt.show()

```

10. Answer

```

import random
def fitness(x):
    return x * x
population = [random.randint(0, 10) for _ in range(6)]
for generation in range(10):
    population.sort(key=fitness, reverse=True)
    parents = population[:2]
    child = (parents[0] + parents[1]) // 2
    if random.random() < 0.2:
        child = random.randint(0, 10)
    population[-1] = child
    print("Generation:", generation + 1)
    print("Best:", max(population, key=fitness))
best = max(population, key=fitness)
print("Best Solution:", best)
print("Fitness:", fitness(best))

```

11. Answer

```

from sklearn.svm import SVC
from sklearn.metrics import accuracy_score
X = [
    [1, 2],
    [2, 3],
    [3, 3],
    [7, 8],
    [8, 9],
    [9, 8]
]
y = [0, 0, 0, 1, 1, 1]

```

```

model = SVC(kernel="linear")
model.fit(X, y)
test = [[5, 5]]
print("Prediction:", model.predict(test))
predictions = model.predict(X)
print("Accuracy:", accuracy_score(y, predictions))
print("Support Vectors:", model.support_vectors_)

```

12. Answer

```

from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
import matplotlib.pyplot as plt
X = [
    [10, 20, 30],
    [12, 24, 35],
    [15, 30, 40],
    [20, 40, 50],
    [25, 50, 60]
]
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)
print("Original Data:")
print(X)
print("Reduced Data:")
print(X_pca)
print("Variance Retained:", pca.explained_variance_ratio_)
plt.scatter(X_pca[:, 0], X_pca[:, 1])
plt.xlabel("PC1")
plt.ylabel("PC2")
plt.show()

```

13. Answer

```

from scipy.cluster.hierarchy import dendrogram, linkage
import matplotlib.pyplot as plt
X = [
    [1, 2],
    [2, 3],
    [3, 2],
    [8, 8],
    [9, 9],
    [8, 9]
]
for method in ["single", "complete", "average"]:
    Z = linkage(X, method=method)
    plt.figure()
    dendrogram(Z)
    plt.title(method + " Linkage")
    plt.show()

```


14. Answer

```
import numpy as np
X = np.array([
    [0, 0],
    [0, 1],
    [1, 0],
    [1, 1]
])
y = np.array([
    [0],
    [1],
    [1],
    [1]
])
np.random.seed(1)
W1 = np.random.rand(2, 3)
W2 = np.random.rand(3, 1)
def sigmoid(x):
    return 1 / (1 + np.exp(-x))
for i in range(10000):
    hidden = sigmoid(np.dot(X, W1))
    output = sigmoid(np.dot(hidden, W2))
    error = y - output
    d_output = error * output * (1 - output)
    d_hidden = d_output.dot(W2.T) * hidden * (1 - hidden)
    W2 += hidden.T.dot(d_output) * 0.1
    W1 += X.T.dot(d_hidden) * 0.1
print("Predicted Output:")
print(output)
print("Classes:")
print((output >= 0.5).astype(int))
```

15. Answer

```
from mlxtend.frequent_patterns import apriori, association_rules
import pandas as pd
transactions = [
    ["Milk", "Bread"],
    ["Milk", "Bread", "Eggs"],
    ["Milk", "Eggs"],
    ["Bread", "Eggs"],
    ["Milk", "Bread", "Eggs"]
]
items = ["Milk", "Bread", "Eggs"]
data = []
for transaction in transactions:
    row = []
    for item in items:
        row.append(item in transaction)
    data.append(row)
```

```
df = pd.DataFrame(data, columns=items)
frequent = apriori(
    df,
    min_support=0.4,
    use_colnames=True
)
print("Frequent Itemsets:")
print(frequent)
rules = association_rules(
    frequent,
    metric="confidence",
    min_threshold=0.6
)
print("Association Rules:")
print(rules[["antecedents", "consequents",
            "support", "confidence", "lift"]])
```